On Singh et. al.'s "Collatz Hash"

Joe Doyle

Trail of Bits

Abstract. Singh et. al. recently uploaded a preprint describing a new hash function inspired by the Collatz Conjecture. After performing some cursory tests, the proposed function appears to be completely unsuitable for cryptographic purposes, and should not be used.

Introduction

Singh et. al. recently uploaded a preprint describing a new hash function inspired by the Collatz Conjecture [SSM25]. The compression function appears to consist of 7 rounds, during which the input block is mixed into the current state, using a Collatz sequence mod 2^{16} to determine the input's bit-rotation amount at each step.

After retrieving the authors' implementation from the GitHub repository linked in the paper, I performed some basic evaluations on the most recent commit in the `main` branch, `ad3ef1df18ba0521801431016b83dc8560edbd92`.

Due to several issues, the source code had to be modified to perform these tests. The modified version is available here, as commit `1b18c22af8a7e485ed1decf158dd8fb3a705ad0f`.

1 Compiler errors, extraneous I/O calls, and undefined behavior

The repository appears to have been developed in a Windows environment, because several `#include` directives use backslashes instead of forward-slashes for paths. Additionally, all three files with `main` functions incorrectly referred to the `Collatz_Hash` function instead of the `Collatz_Hash_6` function defined in `Collatz-Hash.h`. Finally, `Collatz-Hash.h` itself refers to a non-existent variable `m`. Based on context, I replaced this variable `m` with the loop counter `n`.

After fixing these and running `Collatz-Hash-Birthday-Attack`, the executable waits for the user to input the digest size in the terminal, inside the hash function's implementation. Based on this, it appears implausible that the authors ran this particular version for 20 million operations as they claim. I assume they hard-coded the variable `S` (used to determine the digest size, in bytes) to some particular value. I removed the usage of `cin` inside `Collatz_Hash_6` and set `S` to 256. Afterwards, I removed the extraneous debug-printing happening inside the hash function and input padding functions.

After noticing apparent nondeterminism in hash outputs, I recompiled with `-fsanitize=undefined` and discovered that the "..." in section 6.2.1 included an invocation of undefined behavior, shifting a 32 bit value by either 32 or 48: `(X[shuffled32[w]%16]»(48-w%2*16)&0xFFFF)`. I replaced the shift expression with the expression `(16-(w%2)*16)`. When computing the hash of the string "`The quick brown fox jumps over the lazy dog`", the result is

E-mail: joseph.doyle@trailofbits.com (Joe Doyle)

Date of this document: 2025-10-22.

"1955d4a3a215f71b4fc521a09e2759b90b2cd027938f3425cf5ebe969e80f772". This appears to match a prefix of the 384- and 512-bit outputs recorded in table 1, although it differs from the 256-bit result.

In addition to the C++ implementation, a Javascript-based hash calculator is present in the repository. The hash results from that calculator are inconsistent with the C++ implementation and with the table provided in the original paper. In fact, the hash outputs from this calculator do not appear to have a consistent length. This appears to be caused by misusing the Javascript arbitrary-precision `BigInt` type for 64-bit integers, although other issues may also be present. Since this modified C++ implementation matches four of the six example hashes listed in the paper, and the Javascript implementation does not appear to match any of them, I chose to treat this C++ version as the canonical implementation.

2 Evaluation using prefix collision search

Cryptographic hash functions are expected to be pseudorandom, and an n-bit pseudorandom sequence is expected to have a repeated output after $O(2^{n/2})$ steps. Using the `Collatz-Hash-Birthday-Attack.cpp` file but replacing repeated calls to the `mt19937` PRNG with a simple counter, I discovered that the sequence length required to find a prefix collisions was significantly lower than one might expect. The table below shows 40-, 50-, 60-, and 70-bit prefix collisions found solely by incrementing a counter.

input	prefix	hash
"145"	40 bit	3c6ef3763430b0ab63c208039a0d44b05245994d9610699f0dad47d48448955a
"247"	40 bit	3c6ef37634a0a72b7d78bf6e2c940ffb3c6ef37634a0a72b7d78bf6e2c940ffb
"2278"	50 bit	1f83d9afee161d11575f2a9ea7ab01841f83d9afee161d11575f2a9ea7ab0184
"2486"	50 bit	1f83d9afee1635691c75a6b0434662fdfd2992ed90b04d1811fd831b5369cb4e
"68822"	60 bit	1f83d9b09f9b3a666d6b7a3db22b191c57fd01ddff00f54555692fed6bfa0887
"71293"	60 bit	1f83d9b09f9b3a6c3aca69f3cd098838e634d74c47bcc91693ed0dca1136a9ec
"238471"	70 bit	3c6ef3771d117323ef962f627756d7dedf63a168808b57ff295ab77480329866
"302563"	70 bit	3c6ef3771d117323e94207f182e6de6c3c6ef3771d117323e94207f182e6de6c

2.1 Full and nearly-full collisions

While searching for 80-bit collisions, I discovered a range of inputs which contains a full hash collision and several near-collisions. Inputs in this range, and their hashes, are shown below. Note that the first two rows are a full collision.

input string	hash
"1000122"	1f83d9b09f302a150955ac77fc20291aa91a2789609050ce659a082590ead241
"1000123"	1f83d9b09f302a150955ac77fc20291aa91a2789609050ce659a082590ead241
"1000124"	1f83d9aff53fea57f9052c24b6d7105daf0e16190fdda62ae3aa98e34413811b
"1000125"	1f83d9aff53fe9f72473c89416d73aa1da7cb286efddbeae0cab8f8374747c25
"1000126"	1f83d9b0915f1ab59e5f0b211db6088304eca1597cc53786f1f048a2131e1a10
"1000127"	1f83d9b0915f1abb700b5afa52ba088be89fe45d4ce45142546f0ec2b31da89c
"1000128"	1f83d9af945d78cd238e838a5108fc203c6ef376bd738134be93ec16d882bb59
"1000129"	1f83d9af945d78a743199143d104d91b3c6ef376bd7380fbde1ef9cf987ed9be
"1000130"	7291209166205a3b3c6ef376c52e8f957291209166205a3b3c6ef376c52e8f95
"1000131"	99d120906610594d3c6ef376c52e8f5999d120906610594d3c6ef376c52e8f59
"1000132"	c20db9474ca57380b86fbe4a10729e6b4ce95b31053d91769181acab1f307413
"1000133"	a412c91cda1973fee0ee5fd67072d27579a427897d06df36be3c7903c721ad3c
"1000134"	84810a06ed52b1021f83d9b174e708299cecc38a73d28b0852f5ad7db5d33121
"1000135"	7c4663ac4782aae81f83d9b174e707e8c7b9ee51740690f27dc2d844b5e727fb
"1000136"	3c87b2735636e76a72277a2d04d3d768372ac021acec7f00d17af3330d5428bc
"1000137"	267bae604816e6ffcd59c6fe22f27162d70a291c2cec1650f43bac0d16b32a31
"1000138"	bbc86ecf1fa39b5885f47b074a5661e23f9b22ca09832b771f83d9b0b4ec4d6e
"1000139"	fb9856cd38939ab6673abe173ccaf9aff596e67f87a735621f83d9b0b4ec4cb7
"1000140"	e7f8d57c139fcac327c3ee94c2ee8db0e7f8d57c139fcac327c3ee94c2ee8db0
"1000141"	e7f8d57c139fcaba27c3ee93e2ee8db8e7f8d57c139fcaba27c3ee93e2ee8db8
"1000142"	d4029a664964113bd02c13b0fe7d46143c6ef376bd730bcd3526ab24b8142e90

3 Evaluation using `gzip`

Hash function output should not be compressible. I created the file `Collatz-Hash-stream-output.cpp`, shown below, which generates a sequence of 1 million hash outputs in hexadecimal.

```
#include<string>
#include "Collatz Hash/Collatz-Hash.h"

using namespace std;

int main() {
    const int numblocks = 1e6;
    const string key = "testkey";

    for (int i = 0; i < numblocks; ++i) {
        string input = key + to_string(i);
        string fullHash = Collatz_Hash_6(input);
        cout << fullHash;
    }

    return 0;
}
```

The following `bash` command line demonstrates that the resulting output, after being converted to a binary file using xxd, is compressible by another factor of 30.

```
$ { g++ -march=native -O2 -o stream Collatz-Hash-stream-output.cpp &&
    ./stream | xxd -r -p >testbytes;
} && { cat testbytes | gzip -c >testbytes.gz; } && du -hs testbytes{,.gz}
```

```
31M     testbytes
904K    testbytes.gz
```

4 Conclusion

Singh et. al. claim that the "Collatz Hash" hash function "is secure and can be used for password storing and MAC, etc." However, such a strong security claim should be supported by extensive analysis and testing. The very basic analysis performed here shows that it is completely and utterly broken as a hash function, and should not be used for any purpose. I encourage Singh et. al. to withdraw their current preprint and to add warnings to the repository indicating that the algorithm is unsafe.

References

[SSM25] Shaurya Pratap Singh, Bhupendra Singh, and Alok Mishra. Collatz hash: Cryptographic hash algorithm using 3x+1 conjecture. Cryptology ePrint Archive, Paper 2025/1606, 2025. URL: https://eprint.iacr.org/2025/1606.